

ADMISSIO

Plataforma de generación de exámenes de admisión

MEMORIA DESCRIPTIVA

Descripción técnica y funcional del sistema

Versión 1.0 · Noviembre de 2026

https://admissio.camiluxlym.com/

Índice

1. Datos generales 3
2. Resumen 4
3. Antecedentes y problemática 5
4. Objetivos 6
4.1 Objetivo general 6
4.2 Objetivos específicos 6
5. Alcance 7
5.1 Incluye 7
5.2 No incluye (en esta versión) 7
6. Beneficiarios 8
7. Caso de prueba 9
8. Descripción funcional 10
9. Arquitectura del sistema 11
9.1 Procesamiento en el navegador 11
10. Tecnologías utilizadas 12
11. Modelo de datos 13
12. Interfaz de programación (API REST) 14
13. Procesamiento de contenido 15
13.1 Importación sin IA (OCR local) 15
13.2 Importación con IA 15
13.3 Generación de PDF 15
13.4 Distribución de preguntas 15
14. Seguridad y configuración 16
15. Despliegue 17
15.1 Servidor con Docker Compose 17
15.2 Lanzador de escritorio 17
16. Estructura del repositorio 18
17. Adaptación a otras instituciones 19
18. Limitaciones y trabajo futuro 20
19. Conclusiones 21
20. Enlaces 22

1. Datos generales

Concepto	Descripción
Nombre del sistema	Admissio
Tipo	Aplicación web (cliente–servidor) con lanzador de escritorio para Windows
Propósito	Elaboración, gestión y generación de exámenes de admisión
Versión	1.0
Fecha	Noviembre de 2026
Público objetivo	Universidades, institutos de educación superior y centros preuniversitarios
Sistema publicado	https://admissio.camiluxlym.com/
Repositorio	https://github.com/Proyectos-Lino/Proyectos-2026/tree/main/examen-admision
Documentación y entregables	https://drive.google.com/drive/folders/1lVJ96Xk4Wt5RaYXSmIdStM2s4EAWJ62s?usp=sharing

2. Resumen

Admissio es una plataforma web que digitaliza el proceso de elaboración de exámenes de admisión. Centraliza el banco de preguntas, organizado por cursos, y permite definir cuántas preguntas de cada curso corresponden a cada área de admisión y a cada modalidad de ingreso. Con esa configuración genera automáticamente los cuadernillos en PDF, con carátula personalizada y varias versiones (temas) que alteran el orden de las preguntas.

El sistema incluye un editor enriquecido con soporte de fórmulas matemáticas y notación científica, y herramientas de importación masiva desde documentos PDF, texto, imágenes y escáner. La importación puede hacerse con inteligencia artificial o con reconocimiento óptico de caracteres (OCR) local, que funciona sin conexión a servicios externos.

> Para validar la distribución de preguntas por áreas y modalidades se tomó como caso de prueba la estructura de un examen de admisión de la Universidad Nacional San Luis Gonzaga de Ica. Esta referencia es únicamente ilustrativa: Admissio no fue desarrollado por encargo de dicha universidad ni está vinculado a ella. Es un sistema genérico que puede adaptarse a cualquier universidad, instituto o centro preuniversitario.

3. Antecedentes y problemática

En muchas instituciones educativas, los exámenes de admisión se arman de forma manual: se copian preguntas desde distintos documentos, se cuentan a mano las preguntas por curso, se ajusta el formato en un procesador de textos y se preparan varias versiones reordenando el contenido. Este proceso presenta varios problemas:

- **Alto consumo de tiempo** en tareas repetitivas de copiado, formateo y conteo.
- **Errores de distribución**: cursos con más o menos preguntas de las establecidas.
- **Dificultad con la notación especial**: fórmulas, subíndices químicos, ángulos, fracciones y figuras pierden formato al copiarse.
- **Bancos de preguntas dispersos** en archivos sueltos, sin control de uso ni historial.
- **Riesgo de inconsistencias** entre versiones de un mismo examen y sus claves de respuesta.

4. Objetivos

4.1 Objetivo general

Desarrollar un sistema web genérico y adaptable que permita a cualquier institución educativa gestionar su banco de preguntas y generar exámenes de admisión de forma rápida, ordenada y con calidad de impresión.

4.2 Objetivos específicos

- Centralizar las preguntas en un banco organizado por cursos, con búsqueda y filtros.
- Configurar la distribución de preguntas por área de admisión y por modalidad, validando que los totales sean exactos.
- Facilitar la carga masiva de preguntas desde PDF, texto, imágenes y escáner.
- Representar correctamente fórmulas matemáticas, notación química y física, tablas e imágenes.
- Generar cuadernillos en PDF con carátula institucional y múltiples versiones.
- Registrar el historial de exámenes generados y de las acciones realizadas.

5. Alcance

5.1 Incluye

- Gestión de áreas de admisión, cursos y banco de preguntas.
- Editor enriquecido con fórmulas (LaTeX), subíndices, superíndices, símbolos, listas, tablas e imágenes.
- Importación desde PDF/TXT, texto pegado, imágenes (con IA o sin IA) y escáner.
- Configuración de carátulas por combinación de área y modalidad.
- Generación de exámenes en dos pasos, con validación de la distribución y múltiples versiones.
- Historial y descarga de los PDF generados.
- Despliegue con contenedores Docker y lanzador de escritorio para Windows.

5.2 No incluye (en esta versión)

- Inscripción de postulantes ni pagos.
- Calificación automática de hojas de respuestas.
- Gestión de usuarios con roles y permisos (prevista como mejora futura).

6. Beneficiarios

Admissio está pensado para cualquier institución que tome evaluaciones de ingreso:

- **Universidades** públicas y privadas, en sus oficinas o comisiones de admisión.
- **Institutos** de educación superior tecnológica y pedagógica.
- **Centros preuniversitarios** y academias que preparan simulacros.

Como las áreas, cursos, modalidades, cantidades y carátulas son configurables, el sistema no depende de la estructura de ninguna institución en particular.

7. Caso de prueba

Para comprobar el funcionamiento de la distribución de preguntas se cargó una estructura de referencia con áreas de admisión como Ciencias de la Salud, Ciencias e Ingeniería y Ciencias Sociales y Humanidades, dieciséis cursos (Álgebra, Aritmética, Biología, Economía, Educación Cívica, Física, Geografía, Geometría, Historia, Lenguaje, Literatura, Psicología, Química, Razonamiento Matemático, Razonamiento Verbal y Trigonometría) y modalidades de 100, 60 y 40 preguntas, además de una modalidad personalizada.

> Para validar la distribución de preguntas por áreas y modalidades se tomó como caso de prueba la estructura de un examen de admisión de la Universidad Nacional San Luis Gonzaga de Ica. Esta referencia es únicamente ilustrativa: Admissio no fue desarrollado por encargo de dicha universidad ni está vinculado a ella. Es un sistema genérico que puede adaptarse a cualquier universidad, instituto o centro preuniversitario.

Con estos datos se verificó que la suma de la distribución coincide siempre con el total de la modalidad, que la distribución se reescala de forma proporcional cuando se cambia el total, y que los exámenes se generan con el número correcto de preguntas por curso y por versión.

8. Descripción funcional

Módulo	Funciones principales
Dashboard	Indicadores generales (preguntas, áreas, exámenes), gráficos por área e historial de actividades.
Gestión de Áreas	Registro de áreas de admisión, estado activo/inactivo y distribución de cursos por área.
Cursos Globales	Registro de cursos con código, descripción y conteo de preguntas disponibles.
Banco de Preguntas	Alta, edición, activación y búsqueda de preguntas con cinco alternativas (A–E), dificultad y contenido enriquecido.
Importar Preguntas	Extracción desde PDF/TXT, texto pegado, imágenes (con IA o sin IA) y escáner, con revisión a pantalla completa y recorte de figuras.
Configuración	Diseño de carátulas por área y modalidad: institución, logo, título, texto de modalidad y tema inicial, con vista previa y galería.
Generar Exámenes	Asistente de dos pasos: datos del examen (área, modalidad, total, formato, versiones) y distribución por curso con validación del total.
Historial	Consulta por código y área, detalle de versiones y descarga de PDF (individual o en ZIP).

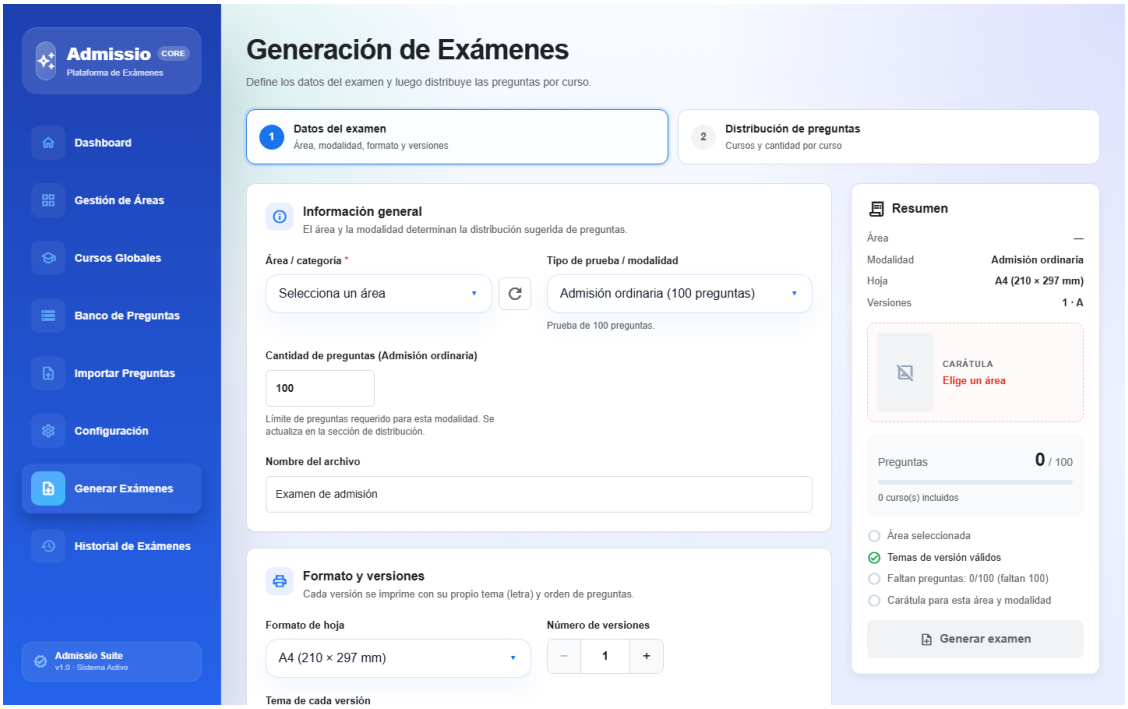


Figura 1. Asistente de generación de exámenes (paso 1) con el panel de resumen.

9. Arquitectura del sistema

Admissio sigue una arquitectura **cliente–servidor de tres capas**, desplegada en contenedores Docker:

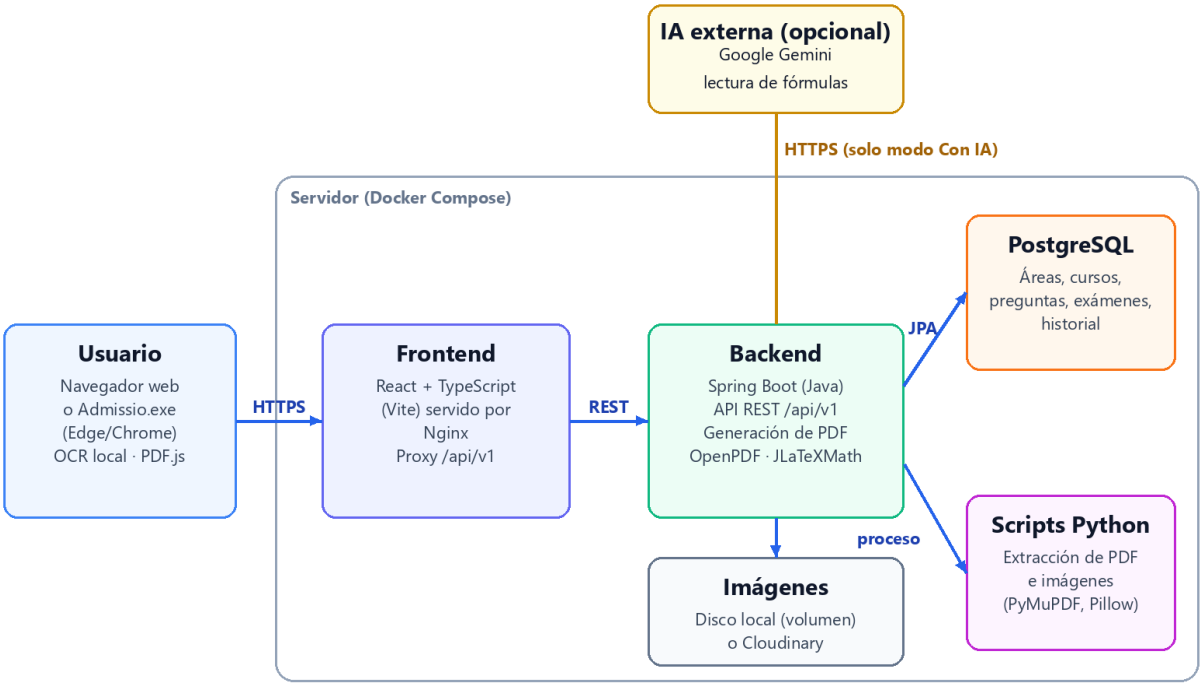


Figura 2. Arquitectura general de Admissio.

- **Capa de presentación (Frontend):** aplicación de una sola página (SPA) en React y TypeScript, compilada con Vite y servida por Nginx. Nginx también actúa como proxy inverso hacia la API, con lo que el navegador accede a todo por un solo dominio.
- **Capa de negocio (Backend):** API REST en Spring Boot (Java) bajo la ruta /api/v1. Aplica las reglas de distribución, arma las versiones y genera los PDF.
- **Capa de datos:** base de datos PostgreSQL, accedida mediante JPA/Hibernate.
- **Servicios de apoyo:** scripts en Python para extraer el contenido de PDF e imágenes; almacenamiento de imágenes en disco local o en Cloudinary; y, de forma opcional, el servicio de IA Google Gemini para leer fórmulas complejas.

9.1 Procesamiento en el navegador

Para reducir la dependencia de servicios externos, varias tareas se ejecutan en la computadora del usuario: la lectura de PDF (PDF.js), el reconocimiento óptico de caracteres (Tesseract.js, con los modelos de idioma incluidos en la aplicación) y la edición de fórmulas (MathLive y KaTeX). Las fuentes, los iconos y los modelos de OCR se sirven desde la propia aplicación, sin depender de redes de distribución de contenido (CDN).

10. Tecnologías utilizadas

Capa	Tecnología	Uso
Frontend	React 19, TypeScript, Vite	Interfaz de usuario (SPA)
Frontend	Tailwind CSS, Radix UI, Material Icons	Estilos, componentes accesibles e iconografía
Frontend	TipTap 3, MathLive, KaTeX	Editor enriquecido y fórmulas matemáticas
Frontend	Tesseract.js, PDF.js	OCR local y lectura de documentos PDF
Frontend	Recharts, TanStack Table, Axios	Gráficos, tablas de datos y llamadas a la API
Backend	Java, Spring Boot 4 (Web MVC, Data JPA, Validation)	API REST y lógica de negocio
Backend	OpenPDF, JLaTeXMath, PDFBox	Generación de PDF con fórmulas
Backend	Python 3 (PyMuPDF, Pillow, google-genai)	Extracción de contenido de PDF e imágenes
Datos	PostgreSQL 17	Base de datos relacional
Infraestructura	Docker, Docker Compose, Nginx	Contenedores, orquestación y servidor web
Escritorio	C# / .NET Framework 4	Lanzador Admissio.exe para Windows

11. Modelo de datos

La base de datos relacional consta de diez tablas principales:

Tabla	Contenido
categoria_examen	Áreas de admisión (nombre, descripción, estado).
curso	Cursos globales (código, nombre, descripción, estado).
categoria*curso*config	Distribución sugerida: cantidad de preguntas de cada curso por área.
pregunta	Preguntas del banco: curso, código, dificultad, contenido enriquecido (JSON) e imagen.
alternativa	Alternativas A–E de cada pregunta, con su contenido y la marca de respuesta correcta.
examen	Exámenes generados: código, nombre, área, modalidad, fecha y estado.
examen*configuracion*curso	Cantidad de preguntas por curso usada en cada examen.
examen_version	Versiones (temas) de cada examen.
examen_pregunta	Preguntas asignadas a cada versión, su orden, puntaje y el orden de sus alternativas.
historial_accion	Registro de auditoría de las acciones realizadas.

El contenido de preguntas y alternativas se guarda como una lista de **bloques JSON** (párrafos, fórmulas LaTeX, listas, tablas e imágenes), lo que permite mostrarlo igual en pantalla y en el PDF.

12. Interfaz de programación (API REST)

El backend expone sus servicios bajo el prefijo **/api/v1**:

Recurso	Operaciones principales
/categorias	Listar, crear, editar y eliminar áreas; administrar su distribución de cursos (/{id}/config-cursos).
/cursos	CRUD de cursos y conteo de preguntas por curso (/conteo-preguntas).
/preguntas	CRUD de preguntas; importación (/importar/imagenes, /importar/pdf-texto, /importar/upload-recorte), extracción y guardado masivo.
/examenes	Generar exámenes, consultar versiones, obtener PDF por versión, solucionario y cambio de tema.
/escaner	Listar escáneres disponibles en el servidor (WIA) y digitalizar hojas (/escanear), con extracción posterior (/extraer).
/imagenes	Servir las imágenes almacenadas localmente.
/dashboard	Estadísticas para el panel principal (/stats).

13. Procesamiento de contenido

13.1 Importación sin IA (OCR local)

El reconocimiento de imágenes sin IA se ejecuta en el navegador y aplica una secuencia de pasos diseñada para material académico:

1. Preprocesamiento: ampliación, escala de grises y binarización (método de Otsu).
2. Doble lectura OCR (página completa y texto disperso) con fusión de resultados.
3. Reconstrucción matemática: detección de barras de fracción y de líneas superiores (periodos, segmentos).
4. Corrección por materia: símbolos químicos validados contra la tabla periódica, notación científica, grados, ángulos (∠) y vocabulario frecuente.
5. Detección de figuras por zonas de tinta fuera del texto, para recortarlas en la revisión.
6. Separación de preguntas y alternativas (incluso alternativas en una misma línea) y unión del texto corrido.

13.2 Importación con IA

De forma opcional, las imágenes se envían al servicio Google Gemini, que devuelve el enunciado, las alternativas y las fórmulas en formato estructurado e indica dónde hay figuras. El uso de IA se limita por un número de consultas y siempre pasa por la revisión humana antes de guardarse.

13.3 Generación de PDF

El backend convierte los bloques de contenido en elementos PDF: texto con formato, fórmulas LaTeX renderizadas como imágenes vectoriales nítidas (JLaTeXMath), tablas, listas e imágenes ubicadas debajo del enunciado. Cada versión recibe su propia carátula y un orden distinto de preguntas.

13.4 Distribución de preguntas

Cuando el usuario modifica el total de preguntas de una modalidad, la distribución por cursos se reescala con el **método del resto mayor**: se asigna a cada curso la parte entera de su proporción y las preguntas restantes se reparten entre los cursos con mayor fracción. Así la suma coincide exactamente con el total, sin superar las preguntas disponibles en el banco.

14. Seguridad y configuración

- Las credenciales (base de datos, Cloudinary, Gemini) se definen como **variables de entorno** y no se guardan en el repositorio. Se incluye un archivo .env.example como plantilla.
- Las imágenes pueden almacenarse en disco local, lo que permite operar sin servicios de terceros.
- El acceso público se realiza por HTTPS a través del proxy inverso.
- Todas las acciones relevantes quedan registradas en el historial de auditoría.

Variable	Descripción
DB_PASSWORD	Contraseña de PostgreSQL.
IMAGENES_ALMACENAMIENTO	local (por defecto) o cloudinary.
CLOUDINARY*CLOUD*NAME / API*KEY* / *API*SECRET	Credenciales de Cloudinary (solo si se usa).
GEMINI*API*KEY	Clave del servicio de IA (solo para el modo Con IA).
VITE*API*BASE_URL	Ruta de la API para el frontend (por defecto /api/v1).

15. Despliegue

15.1 Servidor con Docker Compose

El archivo docker-compose.yml define tres servicios: **admissio-postgres** (base de datos, inicializada con un banco de preguntas de ejemplo), **admissio-backend** (API Spring Boot con Python) y **frontend** (Nginx con la aplicación compilada). Los nombres únicos evitan conflictos con otras aplicaciones del mismo servidor, y Nginx resuelve el backend de forma dinámica.

1. Clonar el repositorio y ubicarse en la carpeta examen-admision.
2. Copiar .env.example a .env y completar las variables.
3. Ejecutar: docker compose up -d --build
4. Publicar el servicio frontend en el dominio deseado (por ejemplo, desde un panel como Dokploy).

15.2 Lanzador de escritorio

La carpeta **Ejecutable** contiene **Admissio.exe**, un lanzador para Windows que no requiere instalación. Comprueba la conexión con el servidor y abre Admissio en una ventana propia de aplicación (Microsoft Edge o Google Chrome en modo aplicación), con un perfil independiente. La dirección del servidor puede cambiarse en el archivo servidor.txt, lo que permite usar el mismo ejecutable con una instalación propia de la institución.

16. Estructura del repositorio

Ruta	Contenido
Backend/	API Spring Boot (src/), scripts Python (scripts/), Dockerfile y pom.xml.
Frontend/	Aplicación React (src/), recursos locales (public/), nginx.conf y Dockerfile.
Preguntas/	Material fuente del banco de preguntas de ejemplo, por curso.
docs/	Manual de usuario, memoria descriptiva y referencia de la API.
Ejecutable/	Admissio.exe, configuración y código fuente del lanzador.
bd*limpia*con_preguntas.sql	Script de creación de la base de datos con datos de ejemplo.
docker-compose.yml	Orquestación de los servicios.
.env.example	Plantilla de variables de entorno.

17. Adaptación a otras instituciones

Admissio no contiene reglas fijas de ninguna institución. Para adaptarlo basta con:

1. Registrar las áreas de admisión y los cursos propios.
2. Definir la distribución de preguntas de cada área y los totales de cada modalidad.
3. Diseñar las carátulas con el nombre y el logo de la institución.
4. Cargar o importar el banco de preguntas.
5. Opcionalmente, desplegar una instancia propia y apuntar el ejecutable a ella mediante servidor.txt.

18. Limitaciones y trabajo futuro

- Incorporar autenticación de usuarios con roles (administrador, docente, revisor).
- Añadir estadísticas de uso de preguntas para evitar repeticiones entre procesos.
- Exportar e importar bancos de preguntas en formatos estándar (por ejemplo, QTI o Moodle XML).
- Integrar la calificación de hojas de respuestas.
- Mejorar el OCR local para diagramas y fórmulas de varios niveles.

19. Conclusiones

Admissio reduce el tiempo y los errores en la elaboración de exámenes de admisión, al centralizar el banco de preguntas, validar automáticamente la distribución por cursos y producir cuadernillos con calidad de impresión y varias versiones. Su diseño configurable y su despliegue en contenedores permiten que cualquier universidad, instituto o centro preuniversitario lo adopte sin modificar el código.

20. Enlaces

Recurso	Enlace
Sistema publicado	https://admissio.camiluxlym.com/
Código fuente	https://github.com/Proyectos-Lino/Proyectos-2026/tree/main/examen-admision
Documentación y entregables	https://drive.google.com/drive/folders/1lVJ96Xk4Wt5RaYXSmIdStM2s4EAWJ62s?usp=sharing